



UNIVERSITY *of* WEST FLORIDA

Machine Learning with Scikit-learn

Convolutional Neural Networks (CNN)

Brian Jalaian, Ph.D.

Associate Professor

Intelligent Systems & Robotics Department

Overview

Convolutional Neural Networks (CNN)

CNN is inspired by how the visual cortex of the human brain works when recognizing objects, and it was developed back to the 1990s by Yann LeCun et al.

- For example, if we're dealing with images, then low-level features, such as edges and blobs, are extracted from the earlier layers, which are combined to form high-level features.
- These high-level features can form more complex shapes, such as the general contours of objects like buildings, cats, or dogs.

A CNN computes feature maps from an input image, where each element comes from a local patch of pixels in the input image.

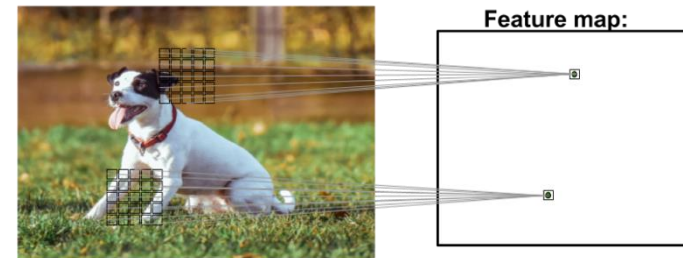


Figure 10.13 Examples of image patches (taken from a validation set) that produce the strongest activation in the hidden units in a network having five convolutional layers trained on ImageNet data. The top nine activations in each feature map are arranged as a 3×3 grid for four randomly chosen channels in each of the corresponding layers. We see a steady progression in complexity with depth, from simple edges in layer 1 to complete objects in layer 5. [From Zeiler and Fergus (2013) with permission.]

Convolutional Neural Networks (CNN)

CNNs will usually perform very well on image-related tasks, and that's largely due to two important ideas:

1. **Sparse connectivity:** A single element in the feature map is connected to only a small patch of pixels.
2. **Parameter sharing:** The same weights are used for different patches of the input image.

Structure of CNN

1. Convolution layers

- Substantially decreases the number of weights (parameters) in the network, and we will see an improvement in the ability to capture salient features.
- In the context of image data, it makes sense to assume that nearby pixels are typically more relevant to each other.

2. Subsampling layers = pooling layers

- Do not have any learnable parameters (no weights, no bias).

3. Fully connected layers

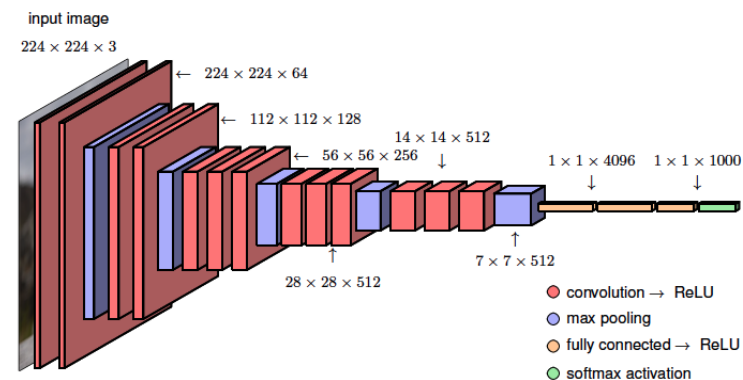


Figure 10.10 The architecture of a typical convolutional network, in this case a model called VGG-16.

Convolution Layer

Discrete convolution in 1D

A **discrete convolution** is mathematically defined as follows:

$$y = x * w \rightarrow y[i] = \sum_{k=-\infty}^{+\infty} x[i - k] w[k]$$

- x: Input
- y: Output
- w: Filter (=Kernel)
- i: Index (runs through each element of the output vector, y)

Math notations

- **subscript**: denote the size of a multidimensional array (tensor)
 - For example, $A_{n_1 \times n_2}$ is a two-dimensional array of size $n_1 \times n_2$.
- **brackets** $[.]$: denote the indexing of a multidimensional array
 - For example, $A[i, j]$ refers to the element at index i, j of matrix A .
- **special symbol** $*$: denote the convolution operation between two vectors or matrices
 - It is not to be confused with the multiplication operator, $*$, in Python.

Padding (= Zero-Padding)

Infinite indices issue

For example, if x has 10 features with indices $0, 1, 2, \dots, 8, 9$, then indices $-\infty: -1$ and $10: +\infty$ are out of bounds for x . Therefore, to correctly compute the summation shown in the preceding formula, it is assumed that x and w are filled with zeros. This will result in an output vector y , that also has infinite size, with lots of zeros as well. Since this is not useful in practical situations, x is padded only with a finite number of zeros.

Solution: Padding (to reduce the size of inputs and outputs)

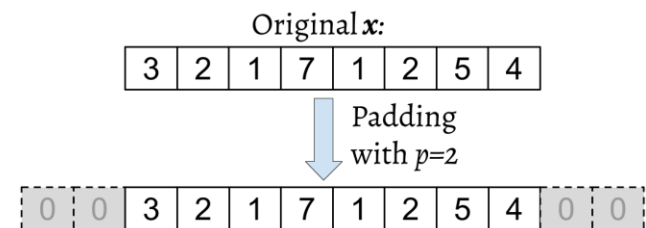
Assume,

- the original input x , has n elements
- the original filter y , has m elements
- $m \leq n$

An example padding of a one-dimensional vector x

- p : number of zeros padded on each side
- padded input x^p , has $n + 2p$ elements

$$y[i] = \sum_{k=-\infty}^{+\infty} x[i - k] w[k]$$



$$y[i] = \sum_{k=0}^{k=m-1} x^p[i + m - k] w[k]$$

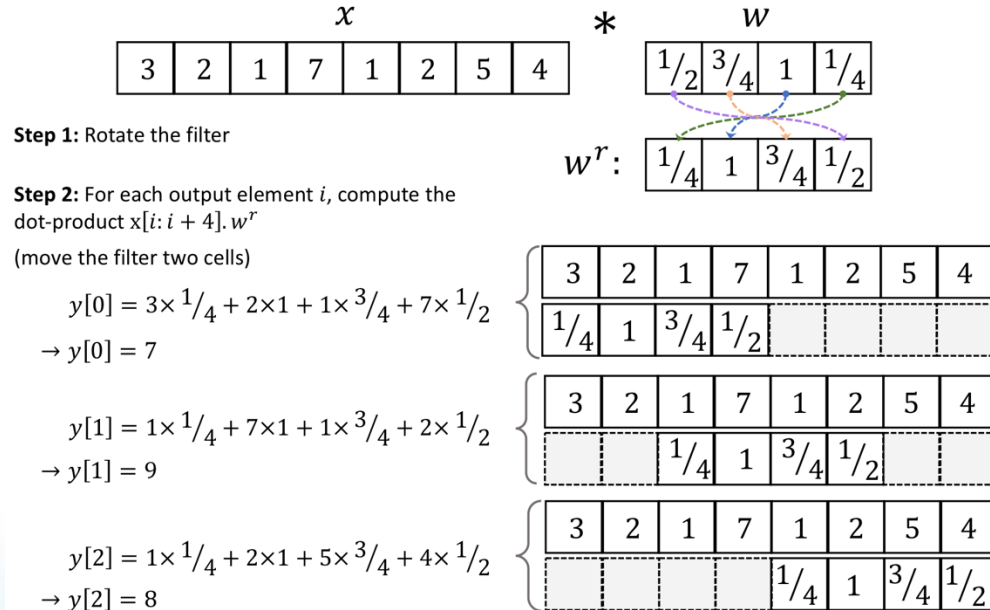
Padding (= Zero-Padding)

Indexing issue

The important point to notice here is that x and w are indexed in different directions in this summation. Computing the sum with one index going in the reverse direction is equivalent to computing the sum with both indices in the forward direction after flipping one of those vectors, x or w , after they are padded.

Solution: Sliding window & rotating vector

- padding size = 0 ($p=0$)
- $s = \text{stride} = 2$
 - This operation is repeated like in a sliding window approach to get all the output elements.
 - The rotated filter, w^r , is shifted by two cells each time we shift.
 - The stride has to be a positive number smaller than the size of the input vector.



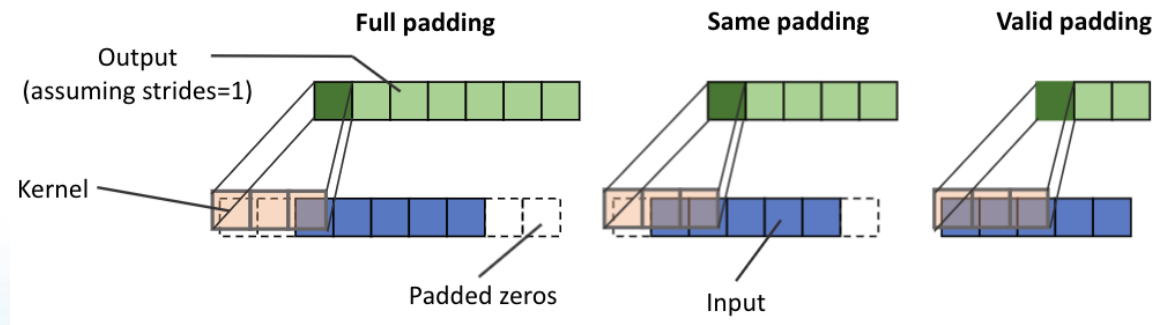
Padding (= Zero-Padding)

The 3 modes of padding

1. Full mode
2. Same mode
 1. p is computed according to the filter size
 2. Used to ensure output size = input size
 3. Most commonly used in CNN since it makes designing a network architecture more convenient.
3. Valid mode
 1. $p=0$ (no padding)
 2. Volume of tensors decrease substantially in multilayer NNs $\rightarrow$ decrease performance

For example,

- input size of 5×5
- kernel size of 3×3
- strides = 1



Size of the convolution output

The output size of a convolution is determined by the total number of times that we shift the filter w along the input vector. Assume,

- input vector size n
- filter vector size m
- padding p
- stride s

Output size:

$$o = \left\lfloor \frac{n + 2p - m}{s} \right\rfloor + 1$$

- $\lfloor \cdot \rfloor$: floor operation
 - It returns the largest integer that is equal to or smaller than the input.
 - For example, $\text{floor}(1.77)=1$

Discrete convolution in 2D

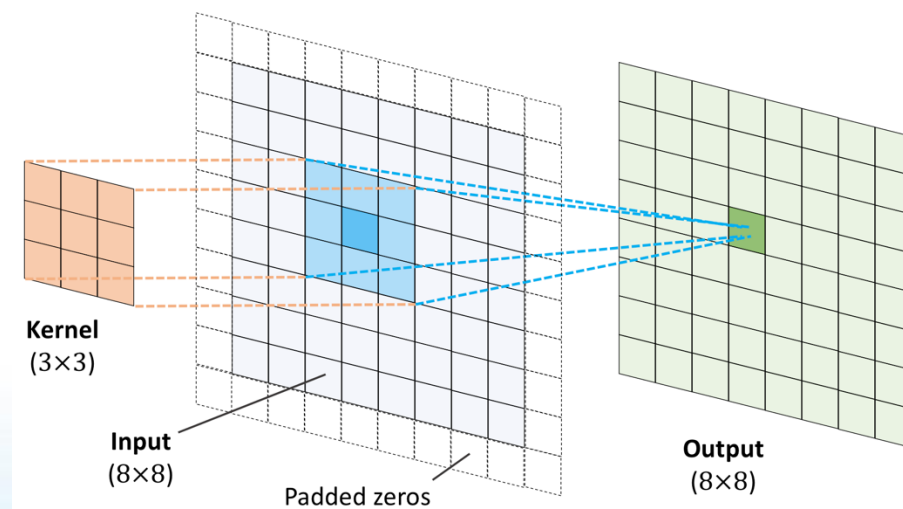
The 2D output is defined mathematically as follows:

$$\mathbf{Y} = \mathbf{X} * \mathbf{W} \rightarrow Y[i, j] = \sum_{k_1=-\infty}^{+\infty} \sum_{k_2=-\infty}^{+\infty} X[i - k_1, j - k_2] W[k_1, k_2]$$

For example,

- input matrix of size 8×8 , padded with zeros with $p=1$
- kernel of size 3×3

The output of the 2D convolution will have a size of 8×8 :



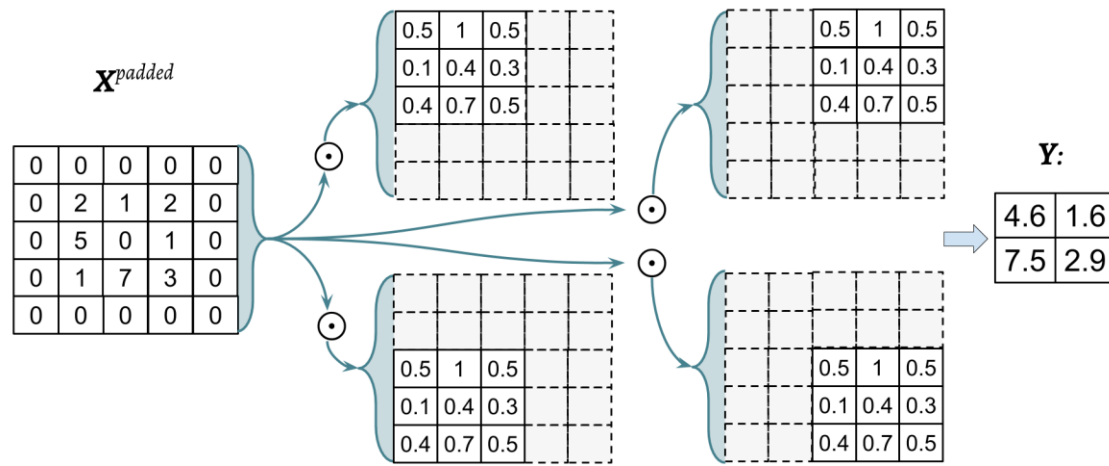
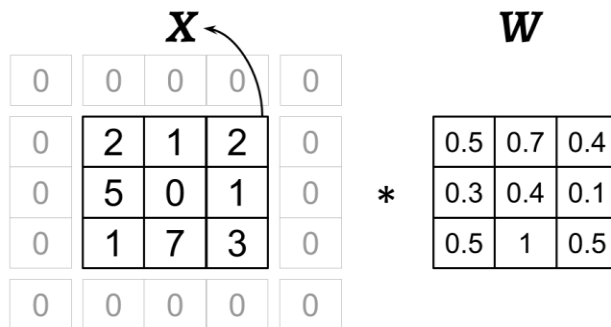
Discrete convolution in 2D - Example

- input matrix: $X_{3 \times 3}$
- filter matrix: $W_{3 \times 3}$
- padding: $p = (1,1)$
- stride: $s = (2,2)$

The padded matrix, $X^{padded}_{5 \times 5}$

$$W^r = \begin{bmatrix} 0.5 & 1 & 0.5 \\ 0.1 & 0.4 & 0.3 \\ 0.4 & 0.7 & 0.5 \end{bmatrix}$$

Shift the rotated filter along the padded input matrix, like a sliding window. Then, compute the sum of the element-wise product, which is denoted by the $\odot$ operator.



Pooling Layer

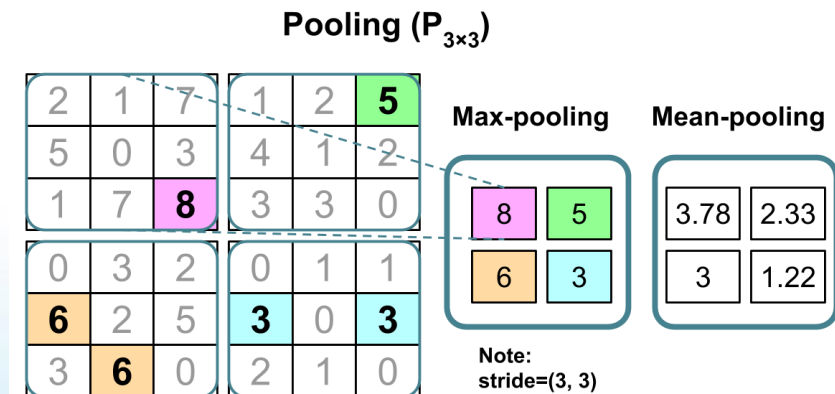
Pooling

Advantages of pooling

- Pooling (max-pooling) introduces a local invariance → Small changes in a local neighborhood do not change the result of max-pooling → Generate features that are more robust to noise in the input data
- Pooling decreases the size of features → Higher computational efficiency → Reduce the degree of overfitting

Overlapping vs non-overlapping pooling

- Traditionally, pooling is assumed to be non-overlapping.
- It can be done by setting: the stride parameter = the pooling size.
 - For example, a non-overlapping pooling layer $P_{n_1 \times n_2}$, requires a stride parameter $s = (n_1, n_2)$.
- Overlapping pooling can be done by:
the stride parameter < the pooling size.



Go to Lecture Code

Let's Implement CNN